

RAG, Knowledge, and Retrieval

Ingestion, indexing, secure filtering, hybrid search, and grounded generation.

Eurskem AI · Agentic Workflow Platform

Generated 2026-08-24

Code revision 8bd16ca4a2a3

Derived from the live repository

Source of truth. Inventories and symbol tables are generated from the current checkout. Narrative explains observed structures and does not replace executable contracts.

Grounded knowledge flow

Documents enter protected object storage, are extracted and chunked, embedded, and indexed with ownership and provenance. Retrieval compiles mandatory security filters separately from model-generated metadata. Hybrid search combines lexical and vector signals; reranking/compression produce cited chunks for RAG generation.

`session_id` /owner and collection scope are mandatory. Selected `document_ids` are server-resolved provenance scope, not generic caller-controlled metadata. Cache identity includes document scope.

File	Responsibility
<code>app/api/knowledge.py</code>	Knowledge Studio collection, profile, index, document and ingestion APIs.
<code>app/ingestion/__init__.py</code>	Python module.
<code>app/ingestion/chunker.py</code>	Structure-aware recursive chunker.
<code>app/ingestion/cli.py</code>	CLI entry point for ingestion.
<code>app/ingestion/collections.py</code>	Per-collection configuration: the controlled vocabulary that makes the platform use-case-agnostic.
<code>app/ingestion/contracts.py</code>	Protocols and the stage registry for the ingestion pipeline.
<code>app/ingestion/coordinator.py</code>	Starts background ingestion work and recovers unfinished jobs.
<code>app/ingestion/embedder.py</code>	Embed chunks into vectors using OpenAI.
<code>app/ingestion/embedding_catalog.py</code>	Embedding models offered when building an Index.
<code>app/ingestion/extractor.py</code>	PDF, Office, plain text, and Markdown extractors.
<code>app/ingestion/indexes.py</code>	Narrow Weaviate index writer used by ingestion.
<code>app/ingestion/jobs.py</code>	Persistent, stage-visible Knowledge Studio ingestion jobs.
<code>app/ingestion/pipeline.py</code>	End-to-end document ingestion pipeline.
<code>app/ingestion/strategies.py</code>	Built-in parser, chunking and enrichment strategies.
<code>app/ingestion/vision.py</code>	Vision-model description of PDF pages carrying figures, charts and tables.
<code>app/knowledge/__init__.py</code>	Knowledge Studio control plane: stable resources, profiles and indexes.
<code>app/knowledge/doc_types.py</code>	The document types a Collection can declare.
<code>app/knowledge/ids.py</code>	Opaque, sortable Knowledge Studio resource identifiers.
<code>app/knowledge/models.py</code>	Versioned Knowledge Studio resource contracts.
<code>app/knowledge/repository.py</code>	Owner-scoped Mongo persistence for Knowledge Studio resources.
<code>app/knowledge/service.py</code>	Authorization-aware operations over Knowledge Studio resources.
<code>app/rag/__init__.py</code>	Saved RAG Agent resources and grounded generation.
<code>app/rag/models.py</code>	Saved RAG Agent query request/response contracts.
<code>app/rag/service.py</code>	Resolve a saved RAG Agent, retrieve securely, then generate an answer.

File	Responsibility
<code>app/retrieval/__init__.py</code>	Hybrid RAG retrieval pipeline.
<code>app/retrieval/compressor.py</code>	Stage 4 of the retrieval pipeline: contextual compression.
<code>app/retrieval/context.py</code>	Parent/sentence-window/contextual expansion and final context assembly.
<code>app/retrieval/contracts.py</code>	Small interfaces for retrieval, fusion, reranking and context expansion.
<code>app/retrieval/filters.py</code>	Typed metadata filters and the non-removable authorization-filter clauses.
<code>app/retrieval/fusion.py</code>	Reciprocal Rank Fusion for independently executed retrieval result sets.
<code>app/retrieval/hybrid_search.py</code>	Stage 2 of the retrieval pipeline: hybrid search with metadata pre-filter.
<code>app/retrieval/models.py</code>	Pydantic models for the retrieval pipeline.
<code>app/retrieval/presets.py</code>	Editable Retrieval Playground starting configurations.
<code>app/retrieval/query_transform.py</code>	Rewrite, multi-query, decomposition, HyDE and validated self-query.
<code>app/retrieval/query_understanding.py</code>	Stage 1 of the retrieval pipeline: query understanding.
<code>app/retrieval/reranker.py</code>	Stage 3 of the retrieval pipeline: LLM-as-judge reranking.
<code>app/retrieval/retriever.py</code>	The retrieval pipeline orchestrator.
<code>app/retrieval/service.py</code>	Canonical retrieval path for workflows, RAG, Playground, MCP and Eval.
<code>app/retrieval/strategies.py</code>	Replaceable dense, sparse and hybrid Weaviate retrieval strategies.
<code>app/retrieval/weaviate_client.py</code>	Weaviate client and schema definition.
<code>ui/src/modes/knowledge/CollectionContext.tsx</code>	Applies a freshly fetched list, keeping the stored choice when it still
<code>ui/src/modes/knowledge/CollectionsPage.tsx</code>	Make the new collection the one every other tab acts on, so the next
<code>ui/src/modes/knowledge/DocumentsIndexesPage.tsx</code>	KnowledgeRoot keys this page by collection, so a switch remounts it with
<code>ui/src/modes/knowledge/IngestionPage.tsx</code>	Job is done — refresh the shared collection so the bar's document and
<code>ui/src/modes/knowledge/KnowledgeRoot.test.tsx</code>	First load is empty; after creating, the list contains the new collection.
<code>ui/src/modes/knowledge/KnowledgeRoot.tsx</code>	Tab content, remounted whenever the selected collection changes so no page can show a previous collection's documents, indexes or results.
<code>ui/src/modes/knowledge/PlaygroundPage.tsx</code>	React/TypeScript module.
<code>ui/src/modes/knowledge/ProfilesAgentsPage.tsx</code>	The chosen answering model IS the Generation Profile, so pick or mint
<code>ui/src/modes/knowledge/TracesPage.tsx</code>	React/TypeScript module.
<code>ui/src/modes/knowledge/collectionStore.ts</code>	The one selected Collection, shared by every Knowledge Studio tab.
<code>ui/src/modes/knowledge/config.ts</code>	React/TypeScript module.
<code>ui/src/modes/knowledge/shared.tsx</code>	React/TypeScript module.

Symbol	Kind	Location	Signature / summary
Chunk	class	app/ingestion/chunker.py:34	One unit of text ready for embedding.
Chunk.embedding_content	method	app/ingestion/chunker.py:59	<code>self</code> The text that should actually be embedded for this chunk.
ChunkConfig	class	app/ingestion/chunker.py:73	Tunable chunking parameters.
_get_tokenizer	function	app/ingestion/chunker.py:96	<code>name: str</code> Return the tokenizer.
_split_sections	function	app/ingestion/chunker.py:121	<code>text: str</code> Split on top-level section headers.
_split_paragraphs	function	app/ingestion/chunker.py:137	<code>text: str</code> Split on blank lines.
_split_sentences	function	app/ingestion/chunker.py:143	<code>text: str</code> Split on sentence boundaries.
_hard_split	function	app/ingestion/chunker.py:152	<code>tokens: list[int], target: int, max_size: int, encoding: tiktoken.Encoding</code> Last-resort: split a token list at fixed boundaries.
_pack_pieces	function	app/ingestion/chunker.py:169	<code>pieces: list[str], encoding: tiktoken.Encoding, config: ChunkConfig</code> Greedy packer.
_split_oversized	function	app/ingestion/chunker.py:220	<code>text: str, encoding: tiktoken.Encoding, config: ChunkConfig</code> Recursive descent: paragraphs → sentences → hard cut.
_apply_overlap	function	app/ingestion/chunker.py:237	<code>chunks: list[str], encoding: tiktoken.Encoding, overlap_tokens: int</code> Prepend the last `overlap_tokens` of chunk N-1 onto the start of chunk N.
chunk_text	function	app/ingestion/chunker.py:261	<code>text: str, config: ChunkConfig None=None</code> Chunk a raw text string.
chunk_document	function	app/ingestion/chunker.py:282	<code>doc: ExtractedDocument, config: ChunkConfig None=None, chunk_id_prefix: str None=None</code> Chunk a full ExtractedDocument into Chunk objects with metadata.
main	function	app/ingestion/chunker.py:368	Quick CLI: `python -m app.ingestion.chunker <path>`.
_parse_metadata	function	app/ingestion/cli.py:18	<code>raw: list[str]</code> Parse repeated --meta key=value flags into a dict.
_amain	async function	app/ingestion/cli.py:29	<code>args: argparse.Namespace</code> Internal helper for the amain step.
main	function	app/ingestion/cli.py:64	Compute the main.
CollectionConfig	class	app/ingestion/collections.py:21	<code>BaseModel</code> Declares one use case's document taxonomy.

Symbol	Kind	Location	Signature / summary
CollectionConfig.validate_ doc_type	method	app/ingestion/ collections.py:35	<code>self, dt: str</code> Raise if 'dt' is not in this collection's declared vocabulary.
CollectionConfig.validate_ doc_types	method	app/ingestion/ collections.py:47	<code>self, dts: list[str]</code> Validate a list (retrieval filters carry a list of doc_types).
CollectionRegistry	class	app/ingestion/ collections.py:53	<code></code> Loads/saves CollectionConfig via the MongoClient's 'collections' accessor — never touches the raw db, matching the project's convention that callers get named collections.
CollectionRegistry.get	method	app/ingestion/ collections.py:67	<code>self, collection_id: str</code> Return the result.
CollectionRegistry.upsert	method	app/ingestion/ collections.py:90	<code>self, cfg: CollectionConfig</code> Upsert the result.
CollectionRegistry.list_all	method	app/ingestion/ collections.py:104	<code>self</code> List the all.
DocumentParser	class	app/ingestion/ contracts.py:17	<code>Protocol</code> Provides the DocumentParser behaviour.
DocumentParser.parse	method	app/ingestion/ contracts.py:19	<code>self, path: Path, *, config: dict[str, Any]</code> No summary declared.
ChunkingStrategy	class	app/ingestion/ contracts.py:22	<code>Protocol</code> Provides the ChunkingStrategy behaviour.
ChunkingStrategy.chunk	method	app/ingestion/ contracts.py:24	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> No summary declared.
ChunkEnricher	class	app/ingestion/ contracts.py:29	<code>Protocol</code> Provides the ChunkEnricher behaviour.
ChunkEnricher.enrich	method	app/ingestion/ contracts.py:31	<code>self, chunks: list[Chunk], *, document: ExtractedDocument, config: dict[str, Any]</code> No summary declared.
EmbeddingProvider	class	app/ingestion/ contracts.py:36	<code>Protocol</code> Provides the EmbeddingProvider behaviour.
EmbeddingProvider.embed	method	app/ingestion/ contracts.py:38	<code>self, texts: list[str]</code> No summary declared.
SearchIndex	class	app/ingestion/ contracts.py:41	<code>Protocol</code> Provides the SearchIndex behaviour.
SearchIndex.index	method	app/ingestion/ contracts.py:43	<code>self, objects: list[dict[str, Any]], vectors: list[list[float]]</code> Write "objects" (with their parallel "vectors") and return the number of objects actually inserted, so the caller can detect a partial write.
UnknownStrategyError	class	app/ingestion/ contracts.py:50	<code>LookupError</code> Exception raised for the UnknownStrategyError case.
StageRegistry	class	app/ingestion/ contracts.py:55	<code></code> Name -> strategy lookup for every ingestion stage kind.

Symbol	Kind	Location	Signature / summary
StageRegistry.parser	method	app/ingestion/ contracts.py:69	<code>self, name: str, instance: DocumentParser</code> Compute the parser.
StageRegistry.chunker	method	app/ingestion/ contracts.py:78	<code>self, name: str, instance: ChunkingStrategy</code> Compute the chunker.
StageRegistry.enricher	method	app/ingestion/ contracts.py:87	<code>self, name: str, instance: ChunkEnricher</code> Compute the enricher.
StageRegistry.get_parser	method	app/ingestion/ contracts.py:96	<code>self, name: str</code> Return the parser.
StageRegistry.get_chunker	method	app/ingestion/ contracts.py:110	<code>self, name: str</code> Return the chunker.
IngestionCoordinator	class	app/ingestion/ coordinator.py:26	 Provides the IngestionCoordinator behaviour.
IngestionCoordinator.submit	method	app/ingestion/ coordinator.py:53	<code>self, job: IngestionJob</code> Schedule "job" to run on the event loop.
IngestionCoordinator.recover	method	app/ingestion/ coordinator.py:60	<code>self</code> Resubmit every job left non-terminal by an interrupted process.
IngestionCoordinator.materialize_sources	method	app/ingestion/ coordinator.py:75	<code>self, job: IngestionJob</code> Download this job's already-uploaded sources to a local temp dir.
IngestionCoordinator.run_safely	method	app/ingestion/ coordinator.py:93	<code>self, job: IngestionJob</code> Run the safely.
EmbedderConfig	class	app/ingestion/ embedder.py:31	 Tunable embedder parameters.
Embedder	class	app/ingestion/ embedder.py:45	 Async OpenAI embedding client.
Embedder.embed	method	app/ingestion/ embedder.py:86	<code>self, texts: list[str]</code> Embed a list of texts.
Embedder.embed_batch_with_retry	method	app/ingestion/ embedder.py:116	<code>self, batch: list[str]</code> Single batch with tenacity-managed retry.
get_embedder	function	app/ingestion/ embedder.py:146	 Lazily-constructed module-level embedder.
EmbeddingModelChoice	class	app/ingestion/ embedding_catalog.py:20	 Provides the EmbeddingModelChoice behaviour.
embedding_model_catalog	function	app/ingestion/ embedding_catalog.py:115	 Compute the embedding model catalog.
_looks_multilingual	function	app/ingestion/ embedding_catalog.py:126	<code>sample: str</code> Internal helper for the looks multilingual step.
select_embedding_model	function	app/ingestion/ embedding_catalog.py:141	<code>*, doc_types: list[str] None=None, document_count: int=0, total_bytes: int=0, sample_text: str=''</code> Deterministically pick an embedding model for a corpus.
ExtractedUnit	class		

Symbol	Kind	Location	Signature / summary
		app/ingestion/ extractor.py:35	One logical unit of an extracted document.
ExtractedDocument	class	app/ingestion/ extractor.py:50	A whole document, broken into ordered units, with metadata.
ExtractedDocument.full_text	method	app/ingestion/ extractor.py:60	<code>self</code> All units joined with double newlines.
Extractor	class	app/ingestion/ extractor.py:69	<code>Protocol</code> Anything callable with <code>.extract(path)</code> returning <code>ExtractedDocument</code> .
Extractor.extract	method	app/ingestion/ extractor.py:72	<code>self, path: Path</code> No summary declared.
PdfExtractor	class	app/ingestion/ extractor.py:78	Extract text from a PDF, one <code>ExtractedUnit</code> per page.
PdfExtractor.extract	method	app/ingestion/ extractor.py:81	<code>self, path: Path</code> Extract the result.
XlsxExtractor	class	app/ingestion/ extractor.py:145	Extract data from an Excel workbook, one <code>ExtractedUnit</code> per worksheet.
XlsxExtractor.extract	method	app/ingestion/ extractor.py:148	<code>self, path: Path</code> Extract the result.
DocxExtractor	class	app/ingestion/ extractor.py:182	Extract text from a Word document.
DocxExtractor.extract	method	app/ingestion/ extractor.py:190	<code>self, path: Path</code> Extract the result.
PptxExtractor	class	app/ingestion/ extractor.py:249	Extract text from a PowerPoint deck, one <code>ExtractedUnit</code> per slide.
PptxExtractor.extract	method	app/ingestion/ extractor.py:256	<code>self, path: Path</code> Extract the result.
PlainTextExtractor	class	app/ingestion/ extractor.py:307	Extract any plain-text file.
PlainTextExtractor.extract	method	app/ingestion/ extractor.py:314	<code>self, path: Path</code> Extract the result.
MarkdownExtractor	class	app/ingestion/ extractor.py:347	Extract text from a Markdown file.
MarkdownExtractor.extract	method	app/ingestion/ extractor.py:357	<code>self, path: Path</code> Extract the result.
DelimitedTextExtractor	class	app/ingestion/ extractor.py:393	Extract rows from a CSV/TSV file, one <code>ExtractedUnit</code> per row block.
DelimitedTextExtractor.extract	method	app/ingestion/ extractor.py:414	<code>self, path: Path</code> Extract the result.
_is_number	function	app/ingestion/ extractor.py:489	<code>value: str</code> Return whether number.
UnsupportedFormatError	class	app/ingestion/ extractor.py:569	

Symbol	Kind	Location	Signature / summary
			<code>ValueError</code> Raised when no extractor is registered for a given file extension.
<code>get_extractor</code>	function	<code>app/ingestion/extractor.py:573</code>	<code>path: Path</code> Return the right extractor for the file's extension.
<code>supported_extensions</code>	function	<code>app/ingestion/extractor.py:583</code>	<code></code> Sorted list of supported file extensions.
<code>main</code>	function	<code>app/ingestion/extractor.py:591</code>	<code></code> Quick CLI: <code>`python -m app.ingestion.extractor path/to/file.<ext>`</code> .
<code>WeaviateSearchIndex</code>	class	<code>app/ingestion/indexes.py:17</code>	<code></code> Satisfies <code>:class:`app.ingestion.contracts.SearchIndex`</code> for one physical Weaviate collection.
<code>WeaviateSearchIndex.ensure_schema</code>	method	<code>app/ingestion/indexes.py:36</code>	<code>self</code> Ensure the schema.
<code>WeaviateSearchIndex.index</code>	method	<code>app/ingestion/indexes.py:43</code>	<code>self, objects: list[dict[str, Any]], vectors: list[list[float]]</code> Compute the index.
<code>IngestionCancelled</code>	class	<code>app/ingestion/jobs.py:31</code>	<code>Exception</code> Exception raised for the IngestionCancelled case.
<code>IngestionJobRunner</code>	class	<code>app/ingestion/jobs.py:36</code>	<code></code> Runs without a second queue platform; progress lives in Mongo.
<code>IngestionJobRunner.embedding_for</code>	method	<code>app/ingestion/jobs.py:69</code>	<code>self, embedding_profile: Any</code> Embedder matching an Embedding Profile, cached per model+dimensions.
<code>IngestionJobRunner.raise_if_cancelled</code>	method	<code>app/ingestion/jobs.py:101</code>	<code>self, job: IngestionJob</code> Raise the if cancelled.
<code>IngestionJobRunner.run</code>	method	<code>app/ingestion/jobs.py:115</code>	<code>self, job: IngestionJob, paths: list[Path], *, metadata: dict[str, Any] None=None</code> Run the result.
<code>IngestionResult</code>	class	<code>app/ingestion/pipeline.py:46</code>	<code></code> What <code>`ingest_file`</code> returns.
<code>ingest_file</code>	async function	<code>app/ingestion/pipeline.py:59</code>	<code>path: Path, metadata: dict[str, Any] None=None, chunk_config: ChunkConfig None=None, object_store: ObjectStore None=None, embedder: Embedder None=None, weaviate: WeaviateClient None=None, mongo: MongoClient None=None, collection_config: CollectionConfig None=None</code> Ingest one document end-to-end.
<code>StandardDocumentParser</code>	class	<code>app/ingestion/strategies.py:16</code>	<code></code> Provides the StandardDocumentParser behaviour.
<code>StandardDocumentParser.parse</code>	method	<code>app/ingestion/strategies.py:18</code>	<code>self, path: Path, *, config: dict[str, Any]</code> Parse the result.
<code>LayoutAwareDocumentParser</code>	class	<code>app/ingestion/strategies.py:31</code>	

Symbol	Kind	Location	Signature / summary
			<code>StandardDocumentParser</code> Use reliable native page/slide/sheet/heading structure when available.
StructureAwareDocumentParser	class	<code>app/ingestion/strategies.py:39</code>	<code>LayoutAwareDocumentParser</code> Provides the StructureAwareDocumentParser behaviour.
StructureAwareDocumentParser.parse	method	<code>app/ingestion/strategies.py:41</code>	<code>self, path: Path, *, config: dict[str, Any]</code> Parse the result.
OcrFallbackDocumentParser	class	<code>app/ingestion/strategies.py:58</code>	<code>StandardDocumentParser</code> Fall back to reading the page as an image when there is no text layer.
OcrFallbackDocumentParser.parse	method	<code>app/ingestion/strategies.py:79</code>	<code>self, path: Path, *, config: dict[str, Any]</code> Parse the result.
VisionAugmentedDocumentParser	class	<code>app/ingestion/strategies.py:140</code>	<code>StandardDocumentParser</code> Add vision-model transcriptions of figures, charts and image tables.
VisionAugmentedDocumentParser.parse	method	<code>app/ingestion/strategies.py:159</code>	<code>self, path: Path, *, config: dict[str, Any]</code> Parse the result.
_chunk_config	function	<code>app/ingestion/strategies.py:213</code>	<code>config: dict[str, Any]</code> Chunk the config.
RecursiveChunkingStrategy	class	<code>app/ingestion/strategies.py:226</code>	 Provides the RecursiveChunkingStrategy behaviour.
RecursiveChunkingStrategy.chunk	method	<code>app/ingestion/strategies.py:228</code>	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
FixedTokenChunkingStrategy	class	<code>app/ingestion/strategies.py:248</code>	 Provides the FixedTokenChunkingStrategy behaviour.
FixedTokenChunkingStrategy.chunk	method	<code>app/ingestion/strategies.py:250</code>	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
StructureAwareChunkingStrategy	class	<code>app/ingestion/strategies.py:295</code>	<code>RecursiveChunkingStrategy</code> Provides the StructureAwareChunkingStrategy behaviour.
StructureAwareChunkingStrategy.chunk	method	<code>app/ingestion/strategies.py:297</code>	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
ParentChildChunkingStrategy	class	<code>app/ingestion/strategies.py:316</code>	<code>RecursiveChunkingStrategy</code> Provides the ParentChildChunkingStrategy behaviour.
ParentChildChunkingStrategy.chunk	method	<code>app/ingestion/strategies.py:318</code>	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
ContextualChunkingStrategy	class	<code>app/ingestion/strategies.py:365</code>	<code>StructureAwareChunkingStrategy</code> Provides the ContextualChunkingStrategy behaviour.

Symbol	Kind	Location	Signature / summary
ContextualChunkingStrategy.chunk	method	app/ingestion/strategies.py:367	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
SentenceWindowChunkingStrategy	class	app/ingestion/strategies.py:389	Provides the SentenceWindowChunkingStrategy behaviour.
SentenceWindowChunkingStrategy.chunk	method	app/ingestion/strategies.py:391	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
_terms	function	app/ingestion/strategies.py:430	<code>text: str</code> Internal helper for the terms step.
SemanticChunkingStrategy	class	app/ingestion/strategies.py:442	Lightweight semantic-boundary strategy using paragraph topic overlap.
SemanticChunkingStrategy.chunk	method	app/ingestion/strategies.py:450	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
MetadataContextEnricher	class	app/ingestion/strategies.py:490	Provides the MetadataContextEnricher behaviour.
MetadataContextEnricher.enrich	method	app/ingestion/strategies.py:492	<code>self, chunks: list[Chunk], *, document: ExtractedDocument, config: dict[str, Any]</code> Compute the enrich.
build_stage_registry	function	app/ingestion/strategies.py:512	Build the stage registry.
PageDescription	class	app/ingestion/vision.py:39	Provides the PageDescription behaviour.
PdfPageVisionDescriber	class	app/ingestion/vision.py:56	Render PDF pages and describe their visual content with a vision model.
PdfPageVisionDescriber.provider	method	app/ingestion/vision.py:76	<code>self</code> The provider.
PdfPageVisionDescriber.model	method	app/ingestion/vision.py:81	<code>self</code> The model.
PdfPageVisionDescriber.available	method	app/ingestion/vision.py:90	<code>self</code> Compute the available.
PdfPageVisionDescriber.unavailable_reason	method	app/ingestion/vision.py:104	<code>self</code> Compute the unavailable reason.
PdfPageVisionDescriber.describe_pages	method	app/ingestion/vision.py:120	<code>self, path: Path, page_indexes: list[int], *, prompt: str None=None</code> Describe the given pages.
PdfPageVisionDescriber._describe	method	app/ingestion/vision.py:176	<code>self, index: int, png: bytes, prompt: str</code> Internal helper for the describe step.
PdfPageVisionDescriber._build_client	method	app/ingestion/vision.py:212	<code>self</code> Build the client.

Symbol	Kind	Location	Signature / summary
<code>_can_render</code>	function	<code>app/ingestion/vision.py:239</code>	<code></code> Return whether render.
<code>_render_pages</code>	function	<code>app/ingestion/vision.py:252</code>	<code>path: Path, indexes: list[int], scale: float</code> Render the given 0-based page indexes to PNG bytes.
<code>pages_with_visual_content</code>	function	<code>app/ingestion/vision.py:273</code>	<code>path: Path, *, min_images: int=1</code> 0-based indexes of PDF pages that carry figures, images or table rules.
<code>DocTypeChoice</code>	class	<code>app/knowledge/doc_types.py:23</code>	<code></code> Provides the DocTypeChoice behaviour.
<code>DocTypeChoice.precision_sensitive</code>	method	<code>app/knowledge/doc_types.py:36</code>	<code>self</code> The precision sensitive.
<code>doc_type_catalog</code>	function	<code>app/knowledge/doc_types.py:67</code>	<code></code> Compute the doc type catalog.
<code>_encode</code>	function	<code>app/knowledge/ids.py:49</code>	<code>value: int, length: int</code> Encode the result.
<code>prefix_for</code>	function	<code>app/knowledge/ids.py:66</code>	<code>kind: str</code> Compute the prefix for.
<code>new_resource_id</code>	function	<code>app/knowledge/ids.py:81</code>	<code>kind: str</code> Mint a fresh opaque identifier for "kind".
<code>scoped_legacy_id</code>	function	<code>app/knowledge/ids.py:90</code>	<code>kind: str, owner_scope_id: str, legacy_key: str</code> Derive a stable identifier for a pre-existing record.
<code>kind_of</code>	function	<code>app/knowledge/ids.py:108</code>	<code>resource_id: str</code> Return the resource kind for "resource_id", or "None" if unknown.
<code>is_kind</code>	function	<code>app/knowledge/ids.py:121</code>	<code>resource_id: str, kind: str</code> Return whether kind.
<code>utcnow</code>	function	<code>app/knowledge/models.py:13</code>	<code></code> Compute the utcnow.
<code>ResourceStatus</code>	class	<code>app/knowledge/models.py:22</code>	<code>str, Enum</code> Enumeration of ResourceStatus values.
<code>ProfileType</code>	class	<code>app/knowledge/models.py:33</code>	<code>str, Enum</code> Enumeration of ProfileType values.
<code>IngestionJobStatus</code>	class	<code>app/knowledge/models.py:46</code>	<code>str, Enum</code> Enumeration of IngestionJobStatus values.
<code>ScopedResource</code>	class	<code>app/knowledge/models.py:61</code>	<code>BaseModel</code> Pydantic model defining the ScopedResource shape.
<code>CollectionResource</code>	class	<code>app/knowledge/models.py:78</code>	<code>ScopedResource</code> Provides the CollectionResource behaviour.
<code>CollectionResource.display_name</code>	method	<code>app/knowledge/models.py:105</code>	<code>self</code> The display name.
<code>DocumentResource</code>	class	<code>app/knowledge/models.py:110</code>	<code>ScopedResource</code> Provides the DocumentResource behaviour.
<code>SourceVersionResource</code>	class	<code>app/knowledge/models.py:136</code>	<code>ScopedResource</code> Provides the SourceVersionResource behaviour.

Symbol	Kind	Location	Signature / summary
ParserProfileConfig	class	app/knowledge/models.py:164	<code>BaseModel</code> Pydantic model defining the ParserProfileConfig shape.
ChunkingProfileConfig	class	app/knowledge/models.py:189	<code>BaseModel</code> Pydantic model defining the ChunkingProfileConfig shape.
ChunkingProfileConfig.validate_token_windows	method	app/knowledge/models.py:218	<code>self</code> Validate the token windows.
EmbeddingProfileConfig	class	app/knowledge/models.py:231	<code>BaseModel</code> Pydantic model defining the EmbeddingProfileConfig shape.
RetrievalProfileConfig	class	app/knowledge/models.py:248	<code>BaseModel</code> Pydantic model defining the RetrievalProfileConfig shape.
RetrievalProfileConfig.validate_counts	method	app/knowledge/models.py:273	<code>self</code> Validate the counts.
RetrievalRoute	class	app/knowledge/models.py:284	<code>BaseModel</code> Pydantic model defining the RetrievalRoute shape.
RetrievalRoutingProfileConfig	class	app/knowledge/models.py:305	<code>BaseModel</code> Pydantic model defining the RetrievalRoutingProfileConfig shape.
RetrievalRoutingProfileConfig.validate_routes	method	app/knowledge/models.py:318	<code>self</code> Validate the routes.
GenerationProfileConfig	class	app/knowledge/models.py:332	<code>BaseModel</code> Pydantic model defining the GenerationProfileConfig shape.
ProfileVersion	class	app/knowledge/models.py:352	<code>ScopedResource</code> Provides the ProfileVersion behaviour.
IndexVersion	class	app/knowledge/models.py:376	<code>ScopedResource</code> Provides the IndexVersion behaviour.
IngestionSourceInput	class	app/knowledge/models.py:413	<code>BaseModel</code> Pydantic model defining the IngestionSourceInput shape.
IngestionJob	class	app/knowledge/models.py:430	<code>ScopedResource</code> Provides the IngestionJob behaviour.
RAGAgentDefinition	class	app/knowledge/models.py:466	<code>ScopedResource</code> Provides the RAGAgentDefinition behaviour.
RetrievalTrace	class	app/knowledge/models.py:492	<code>ScopedResource</code> Provides the RetrievalTrace behaviour.
ResourceNotFoundError	class	app/knowledge/repository.py:41	<code>LookupError</code> A scoped resource lookup found nothing for this owner.
ResourceConflictError	class	app/knowledge/repository.py:45	<code>ValueError</code> A resource exists but is not in a state the caller may act on.
_scope	function	app/knowledge/repository.py:49	<code>owner_scope_id: str, **extra: Any</code> Internal helper for the scope step.
_strip	function		

Symbol	Kind	Location	Signature / summary
		app/knowledge/ repository.py:62	<code>document: dict[str, Any]</code> Internal helper for the strip step.
KnowledgeRepository	class	app/knowledge/ repository.py:75	One instance shared across the app via `services["knowledge_repository"]`.
KnowledgeRepository._collections	method	app/knowledge/ repository.py:88	<code>self</code> The collections.
KnowledgeRepository._documents	method	app/knowledge/ repository.py:93	<code>self</code> The documents.
KnowledgeRepository._source_versions	method	app/knowledge/ repository.py:98	<code>self</code> The source versions.
KnowledgeRepository._profiles	method	app/knowledge/ repository.py:103	<code>self</code> The profiles.
KnowledgeRepository._indexes	method	app/knowledge/ repository.py:108	<code>self</code> The indexes.
KnowledgeRepository._index_documents	method	app/knowledge/ repository.py:113	<code>self</code> The index documents.
KnowledgeRepository._ingestion_jobs	method	app/knowledge/ repository.py:118	<code>self</code> The ingestion jobs.
KnowledgeRepository._rag_agents	method	app/knowledge/ repository.py:123	<code>self</code> The rag agents.
KnowledgeRepository._traces	method	app/knowledge/ repository.py:128	<code>self</code> The traces.
KnowledgeRepository.ensure_indexes	method	app/knowledge/ repository.py:132	<code>self</code> Ensure the indexes.
KnowledgeRepository.create_collection	method	app/knowledge/ repository.py:152	<code>self, resource: CollectionResource</code> Create the collection.
KnowledgeRepository.save_collection	method	app/knowledge/ repository.py:164	<code>self, resource: CollectionResource</code> Save the collection.
KnowledgeRepository.get_collection	method	app/knowledge/ repository.py:181	<code>self, owner_scope_id: str, collection_id: str</code> Return the collection.
KnowledgeRepository.list_collections	method	app/knowledge/ repository.py:196	<code>self, owner_scope_id: str</code> List the collections.
KnowledgeRepository.save_document	method	app/knowledge/ repository.py:209	<code>self, resource: DocumentResource</code> Save the document.
KnowledgeRepository.get_document	method	app/knowledge/ repository.py:226	<code>self, owner_scope_id: str, document_id: str</code> Return the document.
KnowledgeRepository.find_document_by_hash	method	app/knowledge/ repository.py:241	<code>self, owner_scope_id: str, collection_id: str, content_hash: str</code> Find the document by hash.
KnowledgeRepository.list_documents	method	app/knowledge/ repository.py:259	<code>self, owner_scope_id: str, collection_id: str</code> List the documents.
KnowledgeRepository.save_source_version	method	app/knowledge/ repository.py:272	<code>self, resource: SourceVersionResource</code> Save the source version.

Symbol	Kind	Location	Signature / summary
KnowledgeRepository.get_source_version	method	app/knowledge/repository.py:288	<code>self, owner_scope_id: str, source_version_id: str</code> Return the source version.
KnowledgeRepository.next_profile_version	method	app/knowledge/repository.py:308	<code>self, owner_scope_id: str, profile_id: str</code> Compute the next profile version.
KnowledgeRepository.save_profile	method	app/knowledge/repository.py:323	<code>self, profile: ProfileVersion</code> Save the profile.
KnowledgeRepository.get_profile	method	app/knowledge/repository.py:335	<code>self, owner_scope_id: str, profile_id: str, version: int None=None, expected_type: ProfileType None=None</code> Return the profile.
KnowledgeRepository.list_profiles	method	app/knowledge/repository.py:367	<code>self, owner_scope_id: str, profile_type: ProfileType None=None</code> List the profiles.
KnowledgeRepository.save_index	method	app/knowledge/repository.py:389	<code>self, index: IndexVersion</code> Save the index.
KnowledgeRepository.get_index	method	app/knowledge/repository.py:406	<code>self, owner_scope_id: str, index_id: str</code> Return the index.
KnowledgeRepository.list_indexes	method	app/knowledge/repository.py:421	<code>self, owner_scope_id: str, collection_id: str</code> List the indexes.
KnowledgeRepository.save_index_document	method	app/knowledge/repository.py:435	<code>self, *, owner_scope_id: str, index_id: str, document_id: str, source_version_id: str, ingestion_job_id: str, chunk_ids: list[str]</code> Save the index document.
KnowledgeRepository.get_index_document	method	app/knowledge/repository.py:470	<code>self, owner_scope_id: str, index_id: str, document_id: str</code> Return the index document.
KnowledgeRepository.index_document_stats	method	app/knowledge/repository.py:488	<code>self, owner_scope_id: str, index_id: str</code> Return "(document_count, chunk_count)" from authoritative mappings.
KnowledgeRepository.save_ingestion_job	method	app/knowledge/repository.py:503	<code>self, job: IngestionJob</code> Save the ingestion job.
KnowledgeRepository.get_ingestion_job	method	app/knowledge/repository.py:520	<code>self, owner_scope_id: str, ingestion_job_id: str</code> Return the ingestion job.
KnowledgeRepository.list_ingestion_jobs	method	app/knowledge/repository.py:537	<code>self, owner_scope_id: str, collection_id: str None=None</code> List the ingestion jobs.
KnowledgeRepository.list_recoverable_jobs	method	app/knowledge/repository.py:555	<code>self</code> Non-terminal jobs across every owner, for startup recovery only.
KnowledgeRepository.save_rag_agent	method	app/knowledge/repository.py:568	<code>self, agent: RAGAgentDefinition</code> Save the rag agent.
	method		

Symbol	Kind	Location	Signature / summary
KnowledgeRepository.get_rag_agent		app/knowledge/repository.py:585	<code>self, owner_scope_id: str, rag_agent_id: str</code> Return the rag agent.
KnowledgeRepository.list_rag_agents	method	app/knowledge/repository.py:600	<code>self, owner_scope_id: str, search: str None=None</code> List the rag agents.
KnowledgeRepository.save_trace	method	app/knowledge/repository.py:620	<code>self, trace: RetrievalTrace</code> Save the trace.
KnowledgeRepository.get_trace	method	app/knowledge/repository.py:636	<code>self, owner_scope_id: str, retrieval_request_id: str</code> Return the trace.
KnowledgeRepository.list_traces	method	app/knowledge/repository.py:653	<code>self, owner_scope_id: str, limit: int=100</code> List the traces.
KnowledgeRepository.record_manifest_index_history	method	app/knowledge/repository.py:667	<code>self, *, storage_key: str, manifest: dict[str, Any], index_history: dict[str, Any]</code> Upsert the legacy manifest while appending this index's history.
validate_metadata_schema	function	app/knowledge/service.py:26	<code>schema: dict[str, Any]</code> Validate the metadata schema.
workspace_for_scope	function	app/knowledge/service.py:71	<code>owner_scope_id: str</code> Compute the workspace for scope.
KnowledgeService	class	app/knowledge/service.py:83	 Provides the KnowledgeService behaviour.
KnowledgeService.create_collection	method	app/knowledge/service.py:93	<code>self, *, owner_scope_id: str, name: str, description: str='', metadata_schema: dict[str, Any] None=None, doc_types: list[str] None=None</code> Create the collection.
KnowledgeService.create_profile_version	method	app/knowledge/service.py:125	<code>self, *, owner_scope_id: str, profile_type: ProfileType, name: str, strategy: str, config: dict[str, Any], profile_id: str None=None, description: str='', based_on_preset: str None=None</code> Create the profile version.
KnowledgeService.validate_profile_config	method	app/knowledge/service.py:181	<code>profile_type: ProfileType, config: dict[str, Any]</code> Validate the profile config.
KnowledgeService.create_index	method	app/knowledge/service.py:201	<code>self, *, owner_scope_id: str, collection_id: str, parser_profile_id: str, parser_profile_version: int, chunking_profile_id: str, chunking_profile_version: int, embedding_profile_id: str, embedding_profile_version: int</code> Create the index.
KnowledgeService.activate_index	method	app/knowledge/service.py:259	<code>self, *, owner_scope_id: str, collection_id: str, index_id: str</code> Compute the activate index.
	method		

Symbol	Kind	Location	Signature / summary
KnowledgeService.create_rag_agent		app/knowledge/service.py:289	<code>self, *, owner_scope_id: str, name: str, collection_id: str, retrieval_profile_id: str, generation_profile_id: str, routing_profile_id: str None=None, description: str=''</code> Create the rag agent.
KnowledgeService.ensure_default_profiles	method	app/knowledge/service.py:347	<code>self, owner_scope_id: str</code> Ensure the default profiles.
RAGCitation	class	app/rag/models.py:9	<code>BaseModel</code> Pydantic model defining the RAGCitation shape.
RAGQueryResponse	class	app/rag/models.py:34	<code>BaseModel</code> Pydantic model defining the RAGQueryResponse shape.
_build_sources	function	app/rag/service.py:27	<code>citations: list[RAGCitation], chunk_by_id: dict[str, Any]</code> Group citations into one entry per source document (§37 dedup shape).
_build_relevant_context	function	app/rag/service.py:57	<code>chunks: list[Any]</code> One entry per retrieved chunk actually used to ground the answer.
RAGService	class	app/rag/service.py:75	<code></code> Provides the RAGService behaviour.
RAGService._select_route	method	app/rag/service.py:89	<code>self, config: RetrievalRoutingProfileConfig, query: str, llm: Any</code> Select the route.
RAGService.query	method	app/rag/service.py:147	<code>self, *, owner_scope_id: str, rag_agent_id: str, query: str, runtime_filters: dict[str, Any] MetadataFilterGroup None=None, document_ids: list[str] None=None, runtime_context: dict[str, Any] None=None, llm: Any None=None</code> Query the result.
_compress_one	async function	app/retrieval/compressor.py:47	<code>query: str, chunk: RetrievedChunk, llm: LLMGateway, model: str</code> Compress a single chunk.
compress_chunks	async function	app/retrieval/compressor.py:83	<code>query: str, chunks: list[RetrievedChunk], llm: LLMGateway, model: str='claude-haiku-4-5'</code> Compress all chunks in parallel.
GraphContextProvider	class	app/retrieval/context.py:20	<code>Protocol</code> Provides the GraphContextProvider behaviour.
GraphContextProvider.expand	method	app/retrieval/context.py:22	<code>self, chunks: list[RetrievedChunk], *, query: str</code> No summary declared.
expand_context	async function	app/retrieval/context.py:27	<code>chunks: list[RetrievedChunk], *, strategy: str, client: Any, collection_name: str, filters: RetrievalFilters, index_id: str None</code>

Symbol	Kind	Location	Signature / summary
			Apply the requested context-expansion strategy to already-kept chunks.
<code>assemble_context</code>	function	<code>app/retrieval/context.py:67</code>	<code>chunks: list[RetrievedChunk]</code> Build the final "[N] (source: ...)" block passed to generation.
<code>RetrievalStrategy</code>	class	<code>app/retrieval/contracts.py:16</code>	<code>Protocol</code> Provides the RetrievalStrategy behaviour.
<code>FusionStrategy</code>	class	<code>app/retrieval/contracts.py:30</code>	<code>Protocol</code> Provides the FusionStrategy behaviour.
<code>Reranker</code>	class	<code>app/retrieval/contracts.py:37</code>	<code>Protocol</code> Provides the Reranker behaviour.
<code>ContextExpander</code>	class	<code>app/retrieval/contracts.py:44</code>	<code>Protocol</code> Provides the ContextExpander behaviour.
<code>metadata_property_name</code>	function	<code>app/retrieval/filters.py:49</code>	<code>key: str</code> Sanitize a user-supplied metadata key into a safe Weaviate property name.
<code>MetadataFilterValidationError</code>	class	<code>app/retrieval/filters.py:58</code>	<code>ValueError</code> Exception raised for the MetadataFilterValidationError case.
<code>_field_type</code>	function	<code>app/retrieval/filters.py:63</code>	<code>field: str, schema: dict[str, Any]</code> Internal helper for the field type step.
<code>_check_field</code>	function	<code>app/retrieval/filters.py:82</code>	<code>field: str, schema: dict[str, Any]</code> Check the field.
<code>_check_value</code>	function	<code>app/retrieval/filters.py:111</code>	<code>field: str, value: Any, schema: dict[str, Any]</code> Reject a value whose type contradicts the collection's declared schema.
<code>validate_metadata_document</code>	function	<code>app/retrieval/filters.py:134</code>	<code>metadata: dict[str, Any], schema: dict[str, Any]</code> Validate an ingestion-time metadata document against a Collection schema.
<code>_validate_group</code>	function	<code>app/retrieval/filters.py:142</code>	<code>group: MetadataFilterGroup, schema: dict[str, Any]</code> Validate the group.
<code>validate_metadata_filters</code>	function	<code>app/retrieval/filters.py:159</code>	<code>metadata: MetadataFilterGroup None, schema: dict[str, Any]</code> Validate a caller-supplied (or self-query-generated) filter tree.
<code>coerce_metadata_filter_group</code>	function	<code>app/retrieval/filters.py:174</code>	<code>value: dict[str, Any] MetadataFilterGroup None</code> Normalize workflow "runtime_filters"/API filter payloads to one shape.
<code>_predicate_to_filter</code>	function	<code>app/retrieval/filters.py:202</code>	<code>predicate: MetadataFilterPredicate</code> Internal helper for the predicate to filter step.
<code>_group_to_filter</code>	function	<code>app/retrieval/filters.py:235</code>	<code>group: MetadataFilterGroup</code> Group the to filter.
<code>build_secure_where_filter</code>	function	<code>app/retrieval/filters.py:254</code>	<code>filters: RetrievalFilters, *, index_id: str None=None, exclude_parent_chunks: bool=False</code>

Symbol	Kind	Location	Signature / summary
			Compile the non-removable security clause, ANDed with user metadata.
reciprocal_rank_fusion	function	app/retrieval/fusion.py:9	<code>result_sets: list[list[RetrievedChunk]], *, limit: int</code> Fuse independently-ranked result sets (dense+sparse, or per-query multi-query results) into one ranking via Reciprocal Rank Fusion.
deduplicate	function	app/retrieval/fusion.py:36	<code>candidates: list[RetrievedChunk]</code> Drop repeat "chunk_id"s, keeping the highest-scoring occurrence.
_build_where_filter	function	app/retrieval/hybrid_search.py:22	<code>f: RetrievalFilters</code> Translate filters into a Weaviate Filter expression.
hybrid_search	async function	app/retrieval/hybrid_search.py:44	<code>client: Any, embedder: Embedder, collection_name: str, query: str, filters: RetrievalFilters, top_k: int, alpha: float</code> Hybrid BM25 + vector search with metadata pre-filter.
MetadataFilterPredicate	class	app/retrieval/models.py:30	<code>BaseModel</code> Pydantic model defining the MetadataFilterPredicate shape.
MetadataFilterGroup	class	app/retrieval/models.py:46	<code>BaseModel</code> Pydantic model defining the MetadataFilterGroup shape.
RetrievalFilters	class	app/retrieval/models.py:64	<code>BaseModel</code> Metadata pre-filter applied BEFORE hybrid search runs in Weaviate.
RetrievalFilters.user_filter_dump	method	app/retrieval/models.py:90	<code>self</code> Trace-safe view of caller-supplied filters (no security clauses).
RetrievalQuery	class	app/retrieval/models.py:96	<code>BaseModel</code> Single retrieval request.
RetrievedChunk	class	app/retrieval/models.py:131	<code>BaseModel</code> One chunk that survived the pipeline.
RetrievalStage	class	app/retrieval/models.py:176	<code>BaseModel</code> One named step in the staged retrieval pipeline, for the trace.
RetrievalResult	class	app/retrieval/models.py:186	<code>BaseModel</code> Result of one retrieve() call.
_safe_complete	async function	app/retrieval/query_transform.py:21	<code>llm: Any, *, system: str, user: str</code> Internal helper for the safe complete step.
_split_lines	function	app/retrieval/query_transform.py:41	<code>text: str, *, limit: int</code> Split the lines.
transform_query	async function	app/retrieval/query_transform.py:55	<code>request: RetrievalQuery, *, llm: Any, metadata_schema: dict[str, Any]</code> Return "(semantic_query, transformed_queries, generated_filters)".
rewrite_query	async function	app/retrieval/query_understanding.py:22	<code>query: str, llm: LLMGateway</code> Return a search-optimized version of the query.
_RerankScore	class		

Symbol	Kind	Location	Signature / summary
		app/retrieval/reranker.py:33	<code>BaseModel</code> One score for one chunk.
<code>_RerankResponse</code>	class	app/retrieval/reranker.py:40	<code>BaseModel</code> Whole response: a list of scores, one per input chunk.
<code>_format_candidates</code>	function	app/retrieval/reranker.py:65	<code>candidates: list[RetrievedChunk]</code> Build the numbered candidate block for the user prompt.
<code>rerank</code>	async function	app/retrieval/reranker.py:81	<code>query: str, candidates: list[RetrievedChunk], top_n: int, llm: LLMGateway, model: str='gpt-5'</code> Score all candidates in one LLM call, return top_n sorted by score.
<code>retrieve</code>	async function	app/retrieval/retriever.py:30	<code>q: RetrievalQuery, *, weaviate_client: object, llm: LLMGateway, embedder: Embedder, collection_registry=None</code> Run the four-stage retrieval pipeline.
<code>RetrievalAuthorizationError</code>	class	app/retrieval/service.py:39	<code>PermissionError</code> Exception raised for the RetrievalAuthorizationError case.
<code>RetrievalCompatibilityError</code>	class	app/retrieval/service.py:44	<code>ValueError</code> Exception raised for the RetrievalCompatibilityError case.
<code>_safe_candidate</code>	function	app/retrieval/service.py:49	<code>chunk: RetrievedChunk</code> Trace score/provenance, not duplicate raw source text per stage.
<code>RetrievalService</code>	class	app/retrieval/service.py:69	 Provides the RetrievalService behaviour.
<code>RetrievalService.retrieve</code>	method	app/retrieval/service.py:115	<code>self, request: RetrievalQuery, *, owner_scope_id: str, llm: Any None=None</code> Retrieve the result.
<code>RetrievalService._record_failed_trace</code>	method	app/retrieval/service.py:154	<code>self, *, request: RetrievalQuery, request_id: str, owner_scope_id: str, error: Exception, total_ms: float</code> Record the failed trace.
<code>RetrievalService._retrieve_impl</code>	method	app/retrieval/service.py:211	<code>self, request: RetrievalQuery, *, owner_scope_id: str, request_id: str, llm: Any None=None</code> Retrieve the impl.
<code>RetrievalService.compare</code>	method	app/retrieval/service.py:644	<code>self, requests: list[RetrievalQuery], *, owner_scope_id: str, llm: Any None=None</code> Compare the result.
<code>_chunk_from_object</code>	function	app/retrieval/strategies.py:16	<code>obj: Any, *, score_kind: str, rank: int</code> Chunk the from object.
<code>dense_search</code>	async function	app/retrieval/strategies.py:63	<code>*, client: Any, collection_name: str, embedder: Embedder, query: str, filters: RetrievalFilters, index_id: str None, top_k: int, exclude_parent_chunks: bool=False</code> Compute the dense search.

Symbol	Kind	Location	Signature / summary
sparse_search	async function	app/retrieval/strategies.py:106	<code>*, client: Any, collection_name: str, query: str, filters: RetrievalFilters, index_id: str None, top_k: int, exclude_parent_chunks: bool=False</code> Compute the sparse search.
native_hybrid_search	async function	app/retrieval/strategies.py:146	<code>*, client: Any, collection_name: str, embedder: Embedder, query: str, filters: RetrievalFilters, index_id: str None, top_k: int, alpha: float, exclude_parent_chunks: bool=False</code> Compute the native hybrid search.
fetch_chunks_by_id	async function	app/retrieval/strategies.py:197	<code>*, client: Any, collection_name: str, chunk_ids: list[str], filters: RetrievalFilters, index_id: str None=None</code> Fetch the chunks by id.
ensure_collection_schema_on	function	app/retrieval/weaviate_client.py:75	<code>client: weaviate.WeaviateClient, name: str</code> Create "name" with the standard chunk schema if it doesn't exist yet.
upsert_objects_on	function	app/retrieval/weaviate_client.py:100	<code>client: weaviate.WeaviateClient, collection_name: str, objects: list[dict[str, Any]], vectors: list[list[float]]</code> Batch-insert "objects"/"vectors" into "collection_name".
WeaviateClient	class	app/retrieval/weaviate_client.py:141	<code></code> Thin wrapper around weaviate-client v4.
WeaviateClient.connect	method	app/retrieval/weaviate_client.py:157	<code>self</code> Open the gRPC + REST connection.
WeaviateClient.close	method	app/retrieval/weaviate_client.py:191	<code>self</code> Close the underlying gRPC channel.
WeaviateClient.ensure_schema	method	app/retrieval/weaviate_client.py:200	<code>self</code> Create the DocumentChunk collection if it doesn't exist.
WeaviateClient.ensure_collection_schema	method	app/retrieval/weaviate_client.py:226	<code>self, name: str</code> Like :meth:`ensure_schema`, but for an arbitrary physical collection.
WeaviateClient.upsert_objects	method	app/retrieval/weaviate_client.py:231	<code>self, collection_name: str, objects: list[dict[str, Any]], vectors: list[list[float]]</code> Like :meth:`upsert_chunks`, but targeting an arbitrary physical collection.
WeaviateClient.reset_schema	method	app/retrieval/weaviate_client.py:238	<code>self</code> DESTRUCTIVE: delete the collection and recreate it.
WeaviateClient.upsert_chunks	method	app/retrieval/weaviate_client.py:251	<code>self, chunks: list[dict[str, Any]], vectors: list[list[float]]</code> Insert or replace chunks.
WeaviateClient.count_chunks	method	app/retrieval/weaviate_client.py:303	<code>self</code> Total chunks in the collection.
	method		

Symbol	Kind	Location	Signature / summary
WeaviateClient.add_property		app/retrieval/ weaviate_client.py:309	<code>self, prop: Property</code> Non-destructive schema evolution.
get_weaviate_client	function	app/retrieval/ weaviate_client.py:319	 Lazily-constructed module-level client.